Studio Nx

Backend(FastAPI/Python) 80 — http_error_handler() 수정

2026-10-12

Backend(FastAPI/Python) 80 — `http_error_handler()` 수정 (화면은 HTML, API는 JSON)

항목	내용
파일	`studionx/backend/app/core/errors.py`
작업	P1-22 오류 페이지 연결: FastAPI 기본 오류(404, 405 등)를 화면 요청에는 오류 페이지로 응답하기 (PRD 01 FR-116)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	79 `error_page()` · 81 `unexpected_error_handler` 수정 (다음)

1. 하는 일

P1-03에서 만든 처리기에 **화면 요청이면 오류 페이지 HTML**로 응답하는 갈래를 더했습니다. 응답 상태 코드와 `Allow` 같은 헤더는 그대로입니다.

요청	전 (P1-03)	후 (P1-22)
`GET /api/v1/no-such-endpoint` (없는 API)	JSON 404	JSON 404 (그대로)
`POST /api/v1/get-only` (방식이 다름)	JSON 405 + `Allow`	JSON 405 + `Allow` (그대로)
주소창에 입력한 없는 페이지 `/no-such-page`	`SiteFiles`가 404 페이지 (P1-22 이전부터)	그대로
`POST /home.html` (화면 주소에 방식이 다름)	JSON 405	**`errors/400.html`을 상태 405로** (+ `Allow` 헤더)
화면 주소에서 `HTTPException(403)`	JSON 403	**`errors/403.html`을 상태 403으로**
화면 주소에서 `HTTPException(502)`	JSON 502	**`errors/500.html`을 상태 502로**

2. 언제 쓰나

호출하는 곳	용도
`register_exception_handlers()`가 `StarletteHTTPException`에 연결	FastAPI와 Starlette가 직접 낸 오류(없는 주소, 방식이 다름 등)가 날 때마다

3. 코드

import (이번에 추가한 것만)

```
from fastapi.responses import FileResponse, JSONResponse, Response   # Response 추가
```

함수 (바뀐 부분)

```
async def http_error_handler(request: Request, exc: StarletteHTTPException) -> Response:
    if exc.status_code in HTTP_ERRORS:
        code, message = HTTP_ERRORS[exc.status_code]
```

```
    elif exc.status_code >= 500:
        code, message = ErrorCode.INTERNAL_ERROR, "Something went wrong. Please try again."
    else:
        code, message = ErrorCode.BAD_REQUEST, "Bad request."
    if is_api_request(request):
        response = error_json(exc.status_code, code, message)
    else:
        response = error_page(exc.status_code)
    if exc.headers:
        response.headers.update(exc.headers)
    return response
```

바꾼 곳	내용
반환 형식	`JSONResponse`에서 `Response`로 (JSON과 HTML 파일 응답 둘 다 돌려주기 때문입니다)
갈래	`is_api_request(request)` (78)가 참이면 JSON, 거짓이면 `error_page(상태)` (79)
헤더	어느 쪽이든 `exc.headers`를 응답에 붙입니다 (405의 `Allow` 등)

백엔드(Python)는 지금 방식 그대로여서 "최신 문법" 버전은 따로 두지 않습니다.

4. 흐름

1. 오류의 상태 코드로 JSON용 `code`와 `message`를 정합니다 (예전과 같습니다).
2. 요청 주소가 API이면 JSON 응답을 만들고, 화면이면 오류 페이지 응답을 만듭니다.
3. 오류가 가져온 헤더가 있으면 응답에 덧붙입니다.

5. 왜 이렇게

선택	이유
`code`와 `message`를 먼저 구하고 그다음에 갈래	갈래마다 따로 구할 필요가 없고, JSON 쪽의 기존 동작이 한 글자도 바뀌지 않습니다
화면 응답에도 `exc.headers`를 붙임	405 응답은 표준상 `Allow`가 있어야 하며, HTML이어도 같습니다
`exc.detail`을 HTML에 넣지 않음	오류 페이지는 정해진 파일이라서 서버 내부 문구가 섞이지 않습니다
이 처리기만 고치고 `AppError`, 입력 검사 처리기는 그대로	이 두 오류는 `/api/v1`의 라우터 안에서만 나므로, 화면 요청에서는 생기지 않습니다
기존 시험 3개의 주소를 `/api/v1/…`로 옮김	시험용 작은 앱이 `/no-such-api`처럼 API가 아닌 주소를 써서, 이제는 HTML이 돌아오는 것이 맞는 동작이기 때문입니다 (오류 형식을 확인한다는 시험의 목적은 같습니다)

6. 시험

`tests/test_http_error_pages.py` 7개를 새로 만들고, `tests/test_errors.py`는 주소 3곳만 옮겼습니다.

시험	결과
화면 주소의 403	HTML, 상태 403, 본문에 `403`
API 주소의 403	JSON, `error.code`가 `FORBIDDEN`

시험	결과
화면 주소에 POST (405)	HTML(`400.html`), 상태 405, `Allow: GET` 유지
화면 주소의 502	HTML(`500.html`), 상태 502
HTML에 `Traceback`, `HTTPException` 같은 내부 글자가 없음	통과입니다
실제 앱: `POST /home.html`	HTML, 상태 405
실제 앱: 없는 API는 JSON 404, 없는 페이지는 HTML 404	통과입니다

백엔드 pytest 전체 **566개 통과**, `ruff` 통과했습니다.